

What Doesn't Transfer: Generalization-Aware Auditing of Sensitive-Condition Inference from Sanitized LLM-Agent Telemetry

Aditya Jain¹[0009-0007-1837-4448]*, Anmol Pandey¹[0009-0006-2231-1370],
Liang Chen²[0000-0003-3706-6479], and Carsten Maple¹[0000-0002-4715-212X]

¹ University of Warwick, Coventry, United Kingdom
{Aditya.Jain.5,Anmol.Pandey,CM}@warwick.ac.uk

² University of Hertfordshire, Hatfield, United Kingdom
L.CHEN26@herts.ac.uk

Abstract. Observability traces are increasingly used to debug, evaluate and govern large language model (LLM) agents. Even when content is sanitized, operational metadata might reveal whether an interaction belongs to a sensitive rather than a benign condition. A common audit pattern trains a classifier on sanitized telemetry and reads its accuracy as a leakage rate; we argue this measures the wrong thing, and give a generalization-aware methodology making a leakage number depend explicitly on attacker knowledge, the held-out deployment unit and score orientation.

We ask a deliberately scoped question: does the sanitized-metadata residue itself leak once content and the obvious correlated signals (condition-linked tools, topology and workflow choice) are controlled by construction? We answer it in two sequential controlled studies, analyzed separately, of 1,152 and 576 traces over 24 and 6 matched workflows, two models, and two *paired* telemetry stacks. The answer is a scoped boundary: the deployable workflow-unknown attacker does not achieve above-chance cross-workflow inference, and a richer category- and sequence-aware observer does not recover transfer. Removing high-resolution timing collapses the frozen representation to an instrument floor rather than establishing safety. Neither workflow-known training nor one-pair calibration improves the confirmatory result. Secondary analysis identifies one workflow-known kind-aware configuration that warrants powered independent replication before it could be read as an attack. On the same traces and the same observer, the naive audit pattern reports 0.62 AUC where the deployable estimate is 0.45, an overstatement of 0.17 driven mostly by post-hoc score orientation rather than by the split. Separately, workload-wide accuracy can mask narrow, condition-associated reliability anomalies in error telemetry, motivating a distinct screening step. The result is a boundary on this controlled question, not a safety proof; the reusable contribution is the evaluation methodology.

* Corresponding author: Aditya Jain (Aditya.Jain.5@warwick.ac.uk).
Artifact: <https://doi.org/10.5281/zenodo.22743820>

Keywords: LLM agents · observability · privacy side channels · metadata leakage · evaluation methodology.

1 Introduction

LLM agents combine model calls, tools, state, and control logic to complete multi-step tasks. Their non-deterministic execution makes observability operationally valuable, and emerging conventions record model and tool spans, token counts, durations, status values, and parent-child relationships. These records may remain available to platform operators, monitoring vendors and downstream evaluation systems long after the interaction, and as agents are deployed in healthcare triage, financial dispute handling and legal intake, that creates a privacy tension. Content sanitization is the intuitive control: a trace can drop prompts, model responses, tool arguments and outputs, direct identifiers and explicit labels while retaining operational metadata for debugging. But telemetry is retained precisely because it is considered low-risk, so it is kept longer, and seen by a broader population, than the content it describes. Sensitive interactions may take longer, trigger different tools, produce different structures or fail distinctively. If the residual trace nonetheless reveals *whether* an interaction was sensitive, then “we stripped the content” is a weaker control than teams assume.

A convincing answer requires more than a difference inside one workflow. An attack that learns “sensitive is slower” in one workflow may fail when the sign reverses in another, and a classifier evaluated on repeated versions of the same instance may exploit instance-specific regularities rather than a condition signal. The security question is whether a *deployable* attacker can learn an orientation that transfers to the conditions that it will encounter.

This is what our primary metric measures. A classifier produces a score; turning it into a decision requires knowing which end means “sensitive”, and an attacker facing an unseen workflow must fix that direction from its training data, before seeing the target. Fixing the sign from training gives a *signed deployable AUC* in which a value below 0.5 records a real failure mode (the learned direction pointed the wrong way) rather than a success to be recovered by relabeling. The common alternative, $\max(\text{AUC}, 1 - \text{AUC})$, quietly uses target-direction knowledge the attacker lacks at deployment; we use it only as a labeled separability diagnostic. Section 3.4 formalizes this and Section 6 measures what the difference is worth.

Two sequential experiments give a two-part result. The deployable workflow-unknown attacker does *not* transfer across workflows: across 24 workflows and two stacks, signed leave-one-workflow-out AUC sits at or below the chance line, and stronger attacker knowledge does not recover it. Yet the same data contain narrow, condition-associated reliability anomalies that average-case AUC hides: rare output-contract failures in a few cells. These are not statistically confirmed at the independent-replication level, so we report them as exploratory, auditor-side observations. The evaluation discipline that separates these two questions is the reusable contribution.

Research questions. **RQ1:** can a workflow-unknown attacker infer sensitive-condition membership for an *unseen* workflow? **RQ2:** does any such inference survive removing or residualizing high-resolution timing? **RQ3:** does exact workflow knowledge enable inference on unseen semantic instances of that workflow? **RQ4:** can a single labeled sensitive–benign pair fix a useful target orientation? **RQ5:** when workload-wide inference fails, do condition-associated reliability anomalies persist in identifiable cells? Table 5 in Section 8 summarizes the answers.

Contributions.

1. **A generalization-aware evaluation methodology for telemetry-privacy audits.** We make explicit what a leakage number depends on: the attacker’s knowledge (A0/A2/A3), the held-out unit (workflow or semantic instance, never repeated traces), score orientation (signed deployable AUC fixed from training, never post-hoc), and a bounded-null inference rule (bootstrap CI, one-sided test of $\text{AUC} > 0.5$, TOST equivalence). We also disclose and correct a one-pair calibration aggregation error.
2. **A scoped boundary result.** Under matched conditions and the tested observers, the A0 cross-workflow boundary persists under a richer category- and sequence-aware observer with a nonlinear learner, while A2 and A3 do not recover reliable inference under their respective target-workflow knowledge regimes, separating local separability from deployable inference.
3. **A measured price for the naive audit pattern.** On the identical 1,152 traces, with the identical frozen observer and estimator, varying only the audit protocol moves mean workflow AUC from 0.4478 to 0.6175. Section 6 decomposes that 0.1697 gap: the orientation convention accounts for 0.1661 of it and the held-out unit for 0.0195, and the two interact.
4. **A transfer-versus-localization evaluation split.** We separate workload-wide transferable inference from localized, condition-associated reliability anomalies, and show that the two demand different evidence and thresholds. A workload-wide metric at chance can coexist with narrow anomalies that survive only descriptively, which motivates a screening step rather than demonstrating a channel.

2 Background and Related Work

2.1 Agent observability

Agent observability extends conventional tracing to model calls, tool invocations, control-flow decisions, token usage and status values. OpenTelemetry is developing GenAI and agent semantic conventions [1]; AgentTrace proposes structured runtime logging across operational, cognitive and contextual surfaces [2]. Richer traces improve diagnosis and governance but create durable metadata visible to parties who should not learn the interaction’s sensitive condition. We study the residual trace *after* content sanitization; our attacker is denied prompts, responses, tool arguments and outputs, identifiers and labels.

2.2 Privacy side channels in machine-learning systems

Debenedetti et al. show that end-to-end privacy can fail through components around a model even when the model is analyzed in isolation [3]; the trace pipeline is such a component. Recent LLM side-channel work targets inference serving: shared-cache timing recovers or tests private prompts [4,5], output-token count is inferable through latency [6], and topic is inferable from encrypted streaming traffic [7]. We differ in three ways: we study post-sanitization *agent observability* rather than serving behavior or cache probing; the attacker is passive and receives naturally produced traces; and the target is a binary sensitive-versus-benign condition under matched workflows with explicit generalization testing. The broader lesson that empirical privacy evaluations can be misleading unless the attacker model and resampling are stated precisely [8] directly motivates our evaluation discipline.

Fingerprinting and generalization. Website fingerprinting infers hidden activity from packet directions, sizes and timing despite encryption [9], and real-world Tor evaluations show a classifier strong in a closed dataset can lose effectiveness under distribution shift [10]. We adopt that lesson: a security claim must specify what the attacker knows, what unit is held out, and whether orientation can be chosen without target labels.

2.3 Agent privacy and membership inference

A growing line of work evaluates whether agents leak sensitive information through their *outputs and actions*: PrivacyLens measures privacy-norm awareness as models act [11]; AgentDAM evaluates data minimization in autonomous web agents [12]; Privacy in Action studies realistic disclosure and mitigation for LLM-powered agents [13]; and recent work extracts private content from agent *memory* [14]. These target what an agent says, does, or stores; we instead ask whether the residual operational *telemetry*, after content sanitization, supports condition inference. Our threat model is also structurally a membership-inference variant (a binary attacker decision over a held-out unit), and we adopt the methodological lesson of that literature that average-case accuracy can obscure whether an attack reliably succeeds in operationally meaningful regions [15,8], which is why we report signed deployable orientation, equivalence bounds, and explicit generalization units. On the defensive side, telemetry-platform guidance already recommends minimizing, filtering, redacting, or transforming sensitive attributes [16]; our results inform *which* residual attributes most warrant such treatment.

Positioning: what is borrowed, and what is new. The individual principles we use are not ours. Holding out a realistic deployment unit rather than resampling within one, refusing to read average-case accuracy as an attack-success rate, and specifying a claimed attack before measuring it are established practice in website fingerprinting [9,10], membership inference [15], and the critical literature on

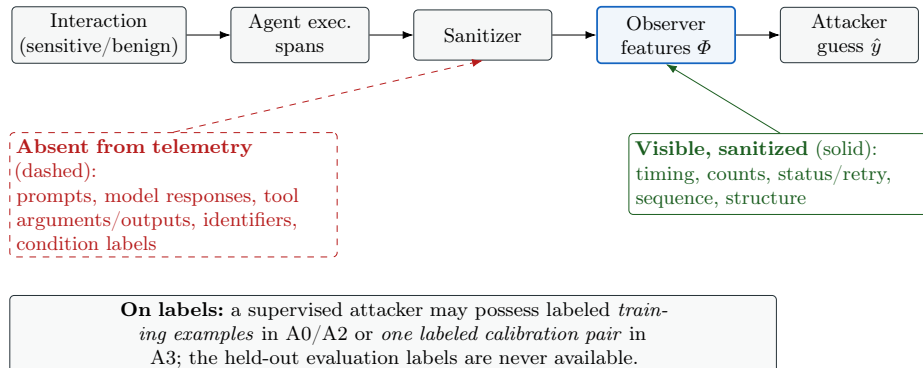


Fig. 1. Threat model. A passive observer of sanitized agent telemetry sees only operational metadata (solid box) and must infer the sensitive condition without the dashed-box content. Condition labels are absent from the telemetry itself; a supervised attacker may possess training labels (A0/A2) or one calibration pair (A3), but never the held-out evaluation labels.

privacy-defense evaluation [8]. Three things here are new: the *composition* of attacker-knowledge regime, held-out unit, score orientation and a bounded-null rule into one procedure an auditor of agent telemetry can run end to end; the *transfer-versus-localization split*, since an audit reporting only workload-wide transfer can miss localized reliability anomalies in the same data; and the *price*, measured on identical traces, of the protocol we criticize (Section 6).

We also narrow a claim. We criticize an audit *pattern* (train on sanitized telemetry, read accuracy as a leakage rate) without attributing it to any specific published agent-telemetry audit: we are aware of none to correct, and the sub-field is young enough that the question has rarely been posed this way. The pattern is documented as a failure mode in the literatures cited above, and our reason to guard against it is demonstrated on our own data rather than asserted about anyone else’s.

3 Threat Model and Security Objective

Figure 1 summarizes the setting: what the sanitizer removes, what survives into the telemetry the attacker sees, and where the condition labels sit relative to both.

3.1 Protected information and observation

Each interaction is sensitive or benign, constructed as matched variants of the same workflow so the intended distinction is semantic condition rather than task topology. The attacker observes sanitized operational features: span/operation counts; model- and tool-call counts as represented by the frozen extractor; exact or

bucketed latency summaries; token-count summaries where retained; status and error counts; sequence length and limited transition summaries; and structural counts (nodes, edges, branches, degree). The exact frozen feature list is given in Section 4.3.

3.2 Exclusions and attacker passivity

The attacker cannot access prompt text, model responses, tool arguments/outputs, direct identifiers as predictive inputs, the label of an uncalibrated target, or the hidden annotations used to construct matched pairs. It is passive: it does not alter execution, submit adaptive probes, control caches or induce failures. We therefore measure condition inference from collected sanitized telemetry, not active cache or chosen-input attacks.

3.3 Attacker-knowledge regimes

A0 (workflow unknown): train on other workflows; the target workflow is held out. **A2 (workflow known):** train on labeled traces from other semantic instances of the known workflow; the target instance is held out. **A3 (one-pair calibration):** the base score is the A0 observer (the same frozen logistic model trained on the other workflows with the identical feature set), and one labeled target pair is used *only* to choose whether to keep or invert that score’s orientation; no parameters are refitted. The remaining five instances are test data. No intermediate regime between A0 and A2 is introduced.

These are not a ladder. A0 is our primary *deployable* regime, defined for an attacker without target-workflow knowledge, and it is the regime our headline claim is about. A2 and A3 are two *different* forms of target-workflow knowledge, five labeled instances versus a single orientation-setting pair, and neither dominates the other; A2 is moreover defined only where semantic-instance identity exists, which is the extension. A0’s failure therefore does not by itself cover the knowledge-rich case, which is why we test A2 and A3 separately and report their limited precision in Section 9.

A telemetry-leakage claim is only meaningful relative to the *dimension held out* at evaluation. We test two: workflow (A0) and semantic instance (A2). Model and stack are robustness checks, not held-out dimensions: we never train on one and evaluate on an unseen one. Environment and time remain future work.

We also note that one-pair calibration (A3) is a single, deliberately noisy point on a calibration curve over k labeled target pairs; whether the boundary is *fundamentally* non-transferable or merely data-hungry is a pre-specifiable future test over $k \in \{0, 1, 2, 4, 8\}$, which we do not run here.

3.4 Security objective and metric

The primary metric is signed deployable AUC: score direction is fixed without consulting held-out target labels, so $\text{AUC} < 0.5$ indicates orientation failure

under the deployment rule, not an invertible success. $\max(\text{AUC}, 1 - \text{AUC})$ would grant post-hoc target-direction knowledge and is used only as an exploratory diagnostic.

A worked micro-example. Train on 23 workflows; if training AUC is 0.58 then $o = +1$ and the raw score is used as-is on the held-out 24th, while 0.41 would set $o = -1$ and evaluate $1 - \text{score}$. Either way the sign is chosen before any held-out label is seen, then frozen. If the held-out workflow separates the other way the signed AUC lands at, say, 0.31: locally informative, globally mis-oriented, and 0.31 is the deployment-consistent value, where $\max(0.31, 0.69) = 0.69$ would require post-hoc knowledge of the target direction. A3 applies the same rule with one labeled pair as its only evidence: that pair fixes o_{wkc} , and $M_{wkc} = \text{AUC}(\{y_i\}, \{o_{wkc}s_i\})$ is scored on the five instances it did not touch, averaged over six rotations (Appendix A).

Decision rules. Two bootstrap intervals appear below, on *different* quantities. (i) The frozen confirmatory rule, for the extension’s positive hypotheses, is on the *difference* from A0: mean gain ≥ 0.05 , Holm-adjusted one-sided $p \leq 0.05$, and a bootstrap lower bound on that difference above *zero*. (ii) The secondary screen of Section 7 is on the *AUC itself*: mean signed AUC ≥ 0.55 with a bootstrap lower bound above 0.5; it is unadjusted and never certifies an attack. The independent unit is the workflow for paired tests and the matched semantic-instance pair for nested resampling.

4 Methodology

4.1 Common design principles

Both experiments use matched sensitive and benign conditions, asking whether the condition itself changes operational behavior. Each scheduled combination is repeated across models, seeds and stacks; collection integrity is checked before analysis and the frozen analysis makes no model calls. The two experiments are sequential studies, not pooled: the descriptive total is 1,728 traces, but each retains its own design, estimand, folds and tests.

4.2 Workflow construct validity

Each workflow is realized as a matched sensitive/benign pair, *controlled synthetic twins*, that share the same frozen workflow path and output schema, so the intended difference is the semantic condition rather than task topology or operational burden. Across the corpus there are 24 distinct ordered operation sequences with operation counts ranging from four to seven. Table 1 gives five representative pairs and what each is matched on.

Table 1. Representative matched workflow pairs (controlled synthetic twins). Each pair shares the same workflow path and output schema; the listed dimensions are held aligned so the remaining difference is the semantic condition.

Workflow	Benign	Sensitive	Matched on (same path & schema)
C03 attendance verification	training attendance record	therapy attendance record	aligned facts (4 low / 8 high complexity), one matched conflict at high complexity
C05 accommodation assessment	shift accommodation plan	pregnancy accommodation plan	requested duration, operational impact, approval route, urgency, evidence count, escalation threshold
C15 hardship appeal	contract hardship appeal	credit hardship appeal	identical five-operation sequence and output contract (see below)
C20 billing dispute	meter-debt dispute	medical-debt dispute	evidence status, disputed amount/stakes, deadline, prior-review state, routing consequence
C22 test-result validation	materials test result	genetic test result	result status, turnaround time, reference-standard lookup, notification requirement, action threshold

C15, for example, follows five fixed operations (FORM intake, RECORD_LOOKUP, EXTERNAL_API validation, CONFLICT_RESOLUTION, SERVICE_ROUTING) identical across both conditions with the same output contract. Matching controls the *presented* administrative facts and workflow mechanics; it does not claim the underlying real-world stakes of the two conditions are equivalent. Prompt length is balanced: over 48 matched pairs per model the median sensitive-minus-benign prompt-token difference is 0 (`qwen2.5:7b` range $[-2, +2]$; `mistral` $[-4, +5]$), so timing separability is not a prompt-length artifact. Exact per-pair counts are in the artifact.

4.3 Frozen observer specification

The confirmatory observer is fully specified and frozen before evaluation. It is a linear classifier (`LogisticRegression`, `class_weight="balanced"`, scikit-learn

defaults; no result-driven tuning), deliberately conservative; Section 7 lifts both representation and learner. Numeric features are vectorized with *no* scaling, imputation, or learned selection; vectorization, fitting, and orientation are fold-local; and workflow ID, semantic-instance ID, model name, and seed are never features. The exact-timing representation has 13 features (four latency summaries plus retry, error, status-error, node, edge, branch, out-degree, sequence-length, and a transition-bigram count); the timing-removed view drops the four latency features, leaving 9. Token counts, though present in the sanitized trace, are *not* among these 13 confirmatory features: prompt-token count enters only as a residualization covariate (Appendix A.1), and token summaries only via the richer observer of Section 7. **Orientation is set from training only** ($o = +1$ if training AUC ≥ 0.5 , else $o = -1$), so a signed test AUC below 0.5 means the training-derived direction failed to transfer; held-out labels never choose orientation. **The two stacks are not independent**: they share prompts, models, seeds, schedule, and host, differing only in orchestration and transport, so they are a paired implementation comparison, not two independent measurements, a distinction that matters in Section 5.3. Exact feature definitions, the bucketed transform, the residualization procedure, software versions, model digests, and per-stack transport detail are in Appendix A.1. For each headline AUC we compute a workflow-level bootstrap 95% CI (10,000 replicates over the held-out unit), a one-sided test of AUC > 0.5 (the only inference a deployable attacker supports), and a TOST equivalence test against chance within ± 0.05 , so that “no signal” is a bounded null rather than a point estimate. Complete confidence intervals, one-sided p -values, and TOST results are provided in the artifact; Appendix C reports the secondary grid’s mean AUCs and abbreviated verdicts, and the key A2/A3 contrasts are reported inline in Section 5.2. The Stack-1-vs-Stack-2 sign-flip statistic tests stack *difference*, not leakage, and is never cited as evidence about leakage.

4.4 Experiment 1: cross-workflow study

24 workflow skeletons; matched sensitive/benign conditions; low/high complexity; two models (`mistral`, `qwen2.5:7b`); three seeds; two stacks; 1,152 accepted traces (one per stack $\times$ workflow $\times$ condition $\times$ complexity $\times$ model $\times$ seed; 48 per workflow, 576 per stack). Evaluation is leave-one-workflow-out, so the observer cannot learn the target’s orientation. Three views: natural timing (confirmatory), timing removed (key secondary), residualized timing (exploratory; Appendix A.1).

4.5 Experiment 2: attacker-knowledge extension

Six workflows selected independently of Experiment 1 performance, each populated with six new matched semantic-instance pairs; high complexity only; two models; two fresh seeds; two stacks; 576 accepted traces (16 per instance), none duplicated, missing, or corrupted. A0 holds out an entire workflow; A2 is leave-one-semantic-instance-out within a known workflow (five train, one test);

Table 2. Experiment 1 leave-one-workflow-out signed deployable AUC. The timing-removed row is an *instrument floor*, not a safety finding: under that deliberately coarse view the representation collapses to a handful of recurring patterns and cannot separate *any* property, so 0.5000 measures the observer rather than the telemetry.

Observer	Stack 1	Stack 2	Mean
Natural timing (confirmatory)	0.4230	0.4725	0.4478
Timing removed (key secondary) [†]	0.5000	0.5000	0.5000
Residualized timing (exploratory)	0.4034	0.4569	0.4301

[†]instrument floor, not a safety result; see text.

A3 rotates each pair as the calibration pair and evaluates on the other five. Views: exact timing (confirmatory), bucketed (descriptive), timing removed.

4.6 Statistical analysis

For A2-vs-A0 and A3-vs-A0 we report mean workflow-level AUC differences, one-sided exact sign-flip tests, Holm correction across the two hypotheses and a nested workflow/pair bootstrap (10,000 replicates), with a success threshold deliberately stronger than nominal significance: gain ≥ 0.05 , adjusted $p \leq 0.05$, positive bootstrap lower bound.

Disclosed correction: one-pair calibration (A3). The frozen A3 implementation averaged the oriented scores across calibration rotations *before* computing AUC, whereas the protocol defines a separate metric per rotation; because AUC is nonlinear in its scores, it did not estimate the stated attacker. Recomputing per rotation (select one calibration pair, evaluate the remaining five, average the six rotation-level AUCs) gives 0.4844 (Stack 1) and 0.4792 (Stack 2), mean 0.4818, an A3–A0 difference of -0.0122 , one-sided $p = 0.7656$ and 95% nested-bootstrap interval $[-0.0715, 0.0479]$. The correction makes calibration slightly worse, does not affect A0 or A2, and changes no conclusion; the formal M_{wkc} definition and the superseded value (0.4957) are in Appendix A.

5 Results by Attacker-Knowledge Regime

5.1 Workflow-unknown (A0): cross-workflow generalization

The primary observer does not learn a direction that transfers: Stack 1 is below chance (0.4230), Stack 2 near chance (0.4725). These signed values must not be inverted after observing test labels. Figure 2 shows why. The orientation-invariant diagnostic averages 0.592 (Stack 1) and 0.604 (Stack 2), with 9 and 11 of the 24 workflows above 0.6 and 3 and 5 above 0.7: many workflows *are* locally separable. Yet once the direction must be fixed in advance from the other 23, 18 of 24 (Stack 1) and 15 of 24 (Stack 2) fall below chance. That distance is exactly the

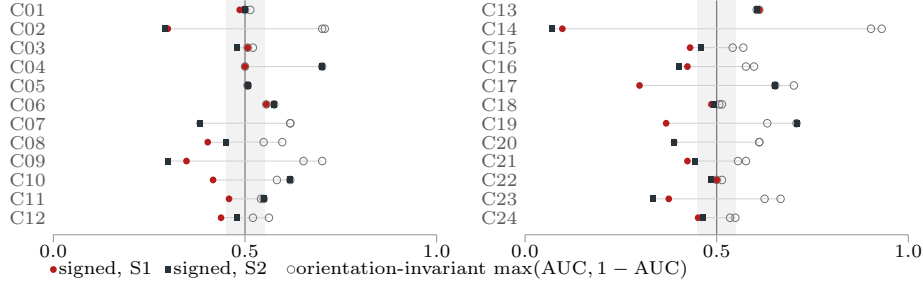


Fig. 2. Per-workflow leave-one-workflow-out AUC, confirmatory natural-timing observer (Experiment 1; C01–C12 left, C13–C24 right). *Filled* markers are the signed deployable values of Table 2: the direction is fixed from the 23 training workflows, so a value left of the chance line records a direction that did not transfer. *Hollow* circles are the orientation-invariant diagnostic, which grants the target direction post hoc and is *not* deployable. The distance between them is what the attacker does not know. Shaded band: ± 0.05 around chance.

knowledge a deployable attacker does not have. The transfer failure is therefore a failure of *orientation*, not an absence of signal: the separating direction is workflow-specific. Per-workflow ranges are in Appendix A. The Stack-1-vs-Stack-2 sign-flip $p = 0.0366$ concerns the mean paired stack difference, not a test against AUC 0.5, and provides no basis for claiming significant leakage.

Timing removed: an instrument floor, not a safety result. Removing timing yields signed AUC exactly 0.5000 for every workflow in both stacks. This is *not* evidence that structural telemetry is safe: the frozen non-timing representation collapses to four to five recurring feature patterns per stack (collision ceiling ≈ 0.5035), near label-balanced except the cell holding the two contract-failure traces. An information-poor representation cannot separate *any* property, so the value is an *instrument floor* that measures the observer rather than the telemetry. It bounds what the aggregate view can claim and motivates the richer observer of Section 7, which tests whether the floor is an artifact of representational poverty.

Residualized timing. Residualizing timing on prompt-token count, model, complexity, and interactions does not improve transfer (0.4034 / 0.4569). This is exploratory because residualization changes the estimand; it supports the reading that recorded nuisance adjustment does not uncover a stable latent condition direction.

5.2 Workflow-known (A2) and one-pair-calibrated (A3)

Exact-timing attacker comparison. Workflow knowledge does not materially improve inference (Table 3). The A2 mean AUC (0.5026) is nominally above chance, but the gain over A0 is only 0.0087, with raw one-sided $p = 0.46875$, Holm $p = 0.9375$, and 95% nested bootstrap $[-0.0781, 0.1042]$: none of the three

Table 3. Experiment 2 signed deployable AUC by attacker and sanitizer view. Only exact-timing A3 is recomputed with the corrected rotation-level metric; the superseded original values are in Appendix A.

Attacker	Stack 1	Stack 2	Mean	Δ from A0
<i>Exact timing (confirmatory)</i>				
A0 (workflow unknown)	0.4809	0.5069	0.4939	—
A2 (workflow known)	0.4878	0.5174	0.5026	+0.0087
A3 (one-pair, corrected)	0.4844	0.4792	0.4818	−0.0122
<i>Other views (A3 not recomputed)</i>				
Bucketed timing, A0 / A2	—		0.5334 / 0.4275	
Timing removed, A0 / A2	—		0.5000 / 0.5000	

success conditions is met. A2 effects are heterogeneous (C20 +0.1771 down to C05 −0.0781), so the one positive cell is a local observation, not a general workflow-known attack. Corrected A3 performs slightly worse than A0, rejecting the hypothesis that one labeled pair stabilizes the score direction.

Sanitizer views. Bucketed timing does not produce a consistent knowledge advantage: A0 rises to 0.5334 but A2 falls to 0.4275, so the single above-chance number (A0 bucketed 0.5334) reverses under workflow knowledge, is unstable across regimes, and does not meet the frozen decision rule of Section 3.4(i) (gain ≥ 0.05 over A0 with the bootstrap lower bound *on that difference* above zero); we report it as descriptive, not a deployable attack. The original bucketed A3 (0.5282) shares the aggregation defect and is reported only for completeness. Timing removal again yields exactly 0.5000.

5.3 Localized condition-associated reliability anomalies

Where they occur. Two cells contain model-output contract failures occurring only under the sensitive condition and recurring across the available seeds: in Experiment 1, four traces in workflow C07, high complexity, **qwen2.5:7b**, at two of three seeds (2 and 3); in Experiment 2, four in workflow C17, instance C17-I03, **mistral**, at both seeds (101 and 202). Because the stacks replay identical prompts, models and seeds, each is reproduced by the paired implementation path rather than independently confirmed. That the locations differ is the point: such failures create local signals contingent on workflow, model and instance rather than a universal signature. Neither supports a generalizable classifier under leave-one-workflow-out evaluation.

Seed-level evidence. The stacks are paired implementation paths, not independent samples (Section 4.3), so the independent unit is the execution seed. At that level C07 fails for two of three sensitive seeds and zero of three benign (one-sided Fisher $p = 0.20$), and C17-I03 for both sensitive seeds and neither benign ($p \approx 0.167$). Neither survives a multiplicity-adjusted scan across the full grid (96

Table 4. The same traces, observer, and estimator under four audit protocols (Experiment 1; mean workflow-level AUC, $n = 24$ workflow units, 95% workflow bootstrap). Only the held-out unit and the orientation convention differ; the random-split cells use five-fold stratified cross-validation. D is the paper’s deployable estimate; N is what the naive pattern would have reported.

Cell	Held-out unit	Orientation	Mean	Stack 1 (95% CI)	Stack 2 (95% CI)
N	random traces	post-hoc	0.6175	0.628 [0.599, 0.659]	0.607 [0.577, 0.644]
O	random traces	signed	0.4514	0.434 [0.381, 0.488]	0.469 [0.422, 0.526]
U	workflow	post-hoc	0.5979	0.592 [0.561, 0.630]	0.604 [0.569, 0.646]
D	workflow	signed	0.4478	0.423 [0.380, 0.461]	0.473 [0.416, 0.526]

qualifying cells in Experiment 1, 72 in Experiment 2) under Benjamini–Hochberg at $q = 0.05$, and recurrence across two seeds does not substitute for that failed correction since the cells were identified from the same data. We treat them as hypothesis-generating anomalies, not channels.

Auditor, not attacker. As an attacker capability these cells would need a cell-indexed rule presupposing the workflow, model and complexity identity the A0/A2 attacker is not given; the localization is an offline auditor method, not a demonstrated attack. Defensively, detailed error telemetry should be screened for condition-associated reliability cells and restricted more strongly than aggregate health metrics, with any discovered cell treated as unconfirmed until independently replicated. Pooling the two paired stacks as if independent would give $p \approx 0.030$ and $p \approx 0.014$, but that treats dependent observations as independent and is not evidence; the seed-level values above are the inferential ones. Full precision/recall and Wilson-interval tables are in the artifact.

6 What the Naive Audit Pattern Reports on the Same Traces

Design. A secondary analysis added after review, specified before it was run, on the *existing* accepted Experiment 1 traces: no recollection, no model calls, no confirmatory result changed. The pattern we criticize has two components, usually adopted together and without comment: the classifier is evaluated on a random split of *traces* (here five-fold stratified cross-validation, fixed in advance), and its score orientation is selected post hoc from the observed test direction. Holding everything else fixed (same traces, representation, estimator and seed) and varying only those two choices gives the 2×2 design of Table 4; cell D is the confirmatory protocol of Table 2, reproduced to six decimal places.

Result. The naive protocol reports 0.6175 where the deployable protocol reports 0.4478: an overstatement of 0.1697 AUC on identical data (0.1221 pooled at the trace level). An auditor running cell N would report a moderate leakage rate and

a defender would reasonably act on it; the deployable answer is that the attack does not work. The decomposition is the informative part, and it was not what we expected. Changing one factor at a time from cell N, holding out the workflow closes 0.0195 of the gap (cell U) while fixing orientation closes 0.1661 (cell O); the two interact and do not sum. On this corpus the orientation convention, not the split, separates an apparent leak from the deployment-consistent estimate.

This balance is corpus-specific. Post-hoc orientation costs nothing when the learned direction is already right; its entire effect is to convert a *transfer failure* into an apparent success by relabeling, and our signed AUCs sit below chance. A corpus whose signed AUC sat above chance would show the reverse, so one corpus and one instantiation of “naive” fix the cost here rather than a universal exchange rate. The lesson is that the two factors interact: post-hoc orientation misleads most precisely where the generalization test is failing, which is where an audit most needs to be trusted.

How much does the fold count matter? The naive cells depend on the number of random folds, one more reason not to read a single such number as a leakage rate. Over $k \in \{3, 5, 10, 20\}$ the overstatement ranges from 0.1697 to 0.1979, and the five folds fixed in advance give the *smallest* of the four, so the quoted figure is the conservative one; cells U and D cannot move with k at all. The sweep is released with the artifact and asserted by its tests.

7 Richer-Observer Secondary Analysis (Category-Aware)

Status and motivation. A *prospectively specified secondary analysis*, not part of the frozen confirmatory study, which is unchanged. The confirmatory observer encodes neither operation-category identity nor ordered structure, so this analysis asks whether the failure to transfer is a property of that limited observer rather than of the telemetry. It runs on the *existing* accepted traces, with no recollection. An *identity-safe* observer using the generic `step.role` category and a *kind-aware* one adding the operation kind are each run under two frozen learners against a descriptive screen fixed in advance; Appendix C gives features, learners, splits and screen in full.

Result: the stronger observer does not transfer on Experiment 1. No configuration recovers cross-workflow transfer. Across both stacks, both observers, all three views and both learners, every one of the 24 A0 cells is inconclusive or statistically equivalent to chance (± 0.05 TOST; mean AUC range 0.435–0.532, overall mean 0.493); none is screen-positive. Adding operation kind changes nothing, so the conclusion holds against a category- and sequence-aware nonlinear adversary substantially stronger than the frozen observer.

Result: scattered, with one repeated screen-positive configuration. The extension is scattered, with one repeated cell: for the workflow-known A2 attacker the kind-aware exact-timing logistic observer reached mean AUC 0.601 ([0.549, 0.653],

Table 5. The whole study in one view: research question, attacker-knowledge regime, sanitizer view, headline signed deployable AUC (mean over the two paired stacks), and the verdict under the decision rules of Section 3.4. Status per Section 4.4: ^c confirmatory, ^s key secondary, ^e exploratory. No row is a confirmed attack.

RQ	Regime	View	Held-out unit	AUC	Verdict
<i>Frozen observer</i>					
RQ1	A0	natural timing ^c	workflow	0.4478	does not transfer
RQ2	A0	timing removed ^s	workflow	0.5000	instrument floor
RQ2	A0	residualized ^c	workflow	0.4301	does not transfer
RQ3	A2	exact timing ^c	sem. instance	0.5026	gain +0.0087; rule not met
RQ4	A3	exact timing ^c	sem. instance	0.4818	−0.0122; calibration fails
<i>Secondary (unadjusted screens; none confirmatory)</i>					
RQ1	A0	richer observer, all views	workflow	0.493	24/24 cells not screen-positive
RQ3	A2	kind-aware, exact timing	sem. instance	0.601/0.613	screen-positive; exploratory
RQ5	auditor	error/status telemetry	cell / seed	—	2 cells; neither survives correction
<i>Same traces, criticized protocol</i> (Section 6)					
—	—	natural timing	random traces	0.6175	overstates by 0.1697

Stack 1) and 0.613 ([0.521, 0.674], Stack 2), the only configuration crossing the screen in *both* paired stacks (its gradient-boosted counterpart crossed only in Stack 1). We treat it as *exploratory*: the grid evaluates many cells with no multiplicity-adjusted test, only six workflow units exist, the stacks are paired rather than independent, and the cell leans on the two most confoundable signals: operation *kind* and exact *timing*. The identity-safe rows, which do not show it, are the conservative reading. The claim narrows rather than reverses: cross-workflow inference is unsupported across all 24 configurations, while workflow-known inference *may* become possible under a kind-aware exact-timing observer and deserves a powered, independently replicated test.

8 Discussion

8.1 What the result establishes

Target-workflow knowledge does not yield reliable *general* inference in the extension: A2 improves mean AUC by only 0.0087 and A3 does not recover orientation,

giving no evidence that workflow identity alone explains the Experiment 1 transfer failure. The boundary is specific to the tested conditions, representation, observers, models, stacks and generalization units (Section 9); the pre-declared richer-observer analysis reduces the concern that it is an artifact of a weak observer.

Why local timing never becomes a deployable attack is visible in Figure 2: within a workflow, AUC is high or low according to whether sensitive traces run slower or faster, and a cross-workflow classifier must learn a *common* direction that never emerges. A3 is the sharpest test, handing the attacker one labeled pair with which to estimate the local direction; one pair did not reliably orient the remaining instances in this experiment.

8.2 Localized reliability anomalies warrant separate auditing

The C07 and C17-I03 cells show a workload-wide classifier at chance while a narrow condition-associated anomaly persists in a few cells. Error and status flags are legible and often broadly accessible, so a platform exposing per-trace failures could surface such an anomaly where no cross-workflow signal exists. Defenders should therefore not read mean AUC as evidence that every field is harmless: screen for condition-associated reliability cells, apply minimum-count thresholds before exposing error analytics, and treat any discovered cell as a secondary finding until independently replicated.

8.3 Implications for telemetry design and evaluation

Four implications follow. (1) *Evaluate at the deployment unit*: the held-out unit must match what the attacker is assumed not to know, since random or repeated-instance splits let a classifier exploit instance-specific regularities. (2) *Report signed, deployable orientation*: fix orientation from training and test against chance with an equivalence bound, because $\max(\text{AUC}, 1 - \text{AUC})$ silently grants the target direction. Section 6 orders these two: on this corpus (2) carries almost all of the correction and (1) very little, and they interact, so an audit adopting (1) without (2) can still report a leak that does not exist. (3) *State attacker knowledge and minimize high-resolution timing*: results change qualitatively across regimes, and exact timing is the feature family most consistently associated with local separability here. (4) *Complement workload-wide accuracy with localization*: an average metric can be at chance while rare anomalies persist.

8.4 What this audit enables

Two follow-on contributions are now pre-specifiable, both requiring new data generation: the kind-aware exact-timing cell that crossed the screen is a confirmatory target, to be frozen and tested on independently collected workflows, another model family and an independent telemetry backend with the decision rule fixed before collection; and the per-feature sensitivity we observe motivates a *telemetry privacy firewall* controlling timing precision, operation-kind exposure and error granularity, evaluated on a privacy–utility frontier.

9 Limitations

Narrow confirmatory observer and excluded features. The confirmatory representation encodes neither operation-category identity nor order, and the emitted parent links are shallow, so usable topology edges are absent by construction; Section 7 addresses that gap but cannot recover a deep execution DAG. Model identity is recorded but is not a classifier feature, so model-conditional boundaries may be missed. **Matched workflows, and what a deployer may not conclude.** Matching isolates the condition effect by holding constant the very channels (condition-correlated workflow choice, tool selection, routing, output schema) likely to carry the most signal in a real deployment, so this is a statement about the metadata *residue* once those channels are closed, close to a defender’s best case. It must not be read as “telemetry does not leak”: where sensitive conditions select different tools or routes, that selection is itself observable and nothing here bounds it. In these experiments residue alone did not suffice, so a practical audit should examine the correlated pathways first. **Extension scope and precision.** Six workflows, six instances each, high complexity only, two fresh seeds. With $n = 6$ generalization units the achievable precision is limited: the A2 exact-timing interval spans $[-0.0781, 0.1042]$, so effects below roughly 0.10 AUC are not resolvable there, and those negative results are consistent with no effect *and* with a small one. **Serving environment.** Both stacks run two approximately 7B open-weight models serially on one host. Batching, queueing, cache contention, cold starts, multi-tenancy and network variance act directly on the latency features carrying most of the local separability here, and collection spanning days or releases would add drift we did not sample; whether the boundary survives a shared commercial serving environment is the open systems question. **Passive attacker, no universal claim.** Active probing, cache manipulation, adaptive testing and auxiliary data are out of scope; chance performance under timing removal is a property of the tested representation, not a proof of zero mutual information.

Artifact Availability

The artifact is at <https://doi.org/10.5281/zenodo.22743820>: frozen schedules and accepted-trace manifests, the raw sanitized span records for both studies, model-identity evidence, every analysis script with its outputs, the frozen protocols and secondary-analysis plans, and SHA-256 hashes for every input. From a clean checkout, a fresh CPython 3.13.6 environment installed from the pinned dependencies reproduces every headline number offline, with no network or model access, via `pwsh repro/run_all.ps1`.

10 Conclusion

We offer a generalization-aware methodology for auditing privacy leakage in sanitized LLM-agent telemetry, making explicit how a leakage number depends

on attacker knowledge, the held-out deployment unit, and score orientation. Across two experiments it yields a boundary: on the 24-workflow corpus a deployable attacker does not transfer, even under a stronger category- and sequence-aware adversary; in the six-workflow extension the pre-specified observer stays at chance, with one secondary kind-aware configuration reaching roughly 0.61 AUC, reported as exploratory pending replication. On the same traces the criticized audit protocol reports 0.6175 where the deployable one reports 0.4478. Detailed error telemetry also showed condition-associated anomalies that averaging hides, in two cells, neither independently confirmed. These conclusions are a boundary, not an assurance, and remain scoped to the tested settings.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Supplementary Material

The following appendices are supplementary. The main paper is complete without them, and reviewers are not required to read them.

A Frozen Observer and Correction Detail

A.1 Frozen observer detail

Estimator. scikit-learn `LogisticRegression` (`class_weight=balanced`, defaults otherwise); the only study difference is the iteration cap/seed (`max_iter` 1000, `random_state` 20260624 for Experiment 1; 2000/20260627 for Experiment 2). Features are vectorized with `DictVectorizer(sparse=False)`; missing required features are an integrity failure, not imputed. **Exact feature list (13).** `lat_{mean,std,max,sum}`, `retry_sum`, `error_count`, `status_error_count`, `node_count`, `edge_count`, `branch_count`, `max_out_degree`, `sequence_length`, `distinct_transition_bigrams`; the timing-removed view drops the four `lat_*` features. **Bucketed view:** each non-negative timing $x \mapsto \lfloor \log_2(1+x) \rfloor$, capped at 20, boundaries fixed in advance. **Residualized (exploratory):** each timing feature is regressed (fold-local OLS, `numpy.linalg.lstsq`) on an intercept, prompt-token count, model and complexity indicators, and two prompt-token interactions, fitted on training workflows with labels unused; prompt-token count is not then a classifier feature. **Stacks.** Stack 1 is a LangGraph `StateGraph` with LangChain `ChatOllama` (HTTPX); Stack 2 a custom deterministic executor with direct `http.client` to the same Ollama endpoint; both emit spans through separately implemented in-memory OpenTelemetry exporters under a serial policy.

A.2 Software and model identity

CPython 3.13.6, scikit-learn 1.9.0, NumPy 2.4.6, SciPy 1.17.1, LangGraph 1.2.5, LangChain 1.3.9, langchain-ollama 1.1.0, OpenTelemetry API/SDK 1.42.1, Ollama server 0.18.2; a frozen dependency lock is released with the artifact. Ollama models are pinned by manifest and weight-layer digest rather than by tag: `qwen2.5:7b` manifest SHA-256 `845dbda0...a0b697e`, primary weight layer `sha256:2bada8a7...33730`. The mutable `mistral:latest` tag was resolved once and pinned to manifest SHA-256 `6577803a...fbc1cf`, primary weight layer `sha256:f5074b12...a2b31f`. Full blob sets are in the artifact’s `MODEL_IDENTITY_LOCK.json`.

A.3 A3 correction detail

For workflow w , stack k , calibration pair c with base scores s_i , the labeled pair fixes an orientation $o_{wkc} \in \{+1, -1\}$ and the protocol-consistent metric is $M_{wkc} = \text{AUC}(\{y_i\}, \{o_{wkc}s_i\})$ over the five non-calibration instances, averaged across rotations then across workflows/stacks. The frozen implementation instead averaged the five oriented scores assigned to each trace and computed one AUC from the averaged scores. Because AUC is nonlinear in the scores, averaging scores before the metric is not equivalent to averaging rotation-level AUCs, so the implementation did not estimate the protocol attacker. The corrected per-rotation recomputation gives Stack 1 0.4844, Stack 2 0.4792, mean 0.4818; the superseded original value (0.4957) is retained here only. The correction script and rotation-level metrics are archived with the artifact.

A.4 Supplementary result detail

Experiment 1 workflow-level AUC ranges: Stack 1 0.0972–0.6111, Stack 2 0.0694–0.7083. The superseded original A3 value (0.4957, defective aggregation) is recorded for audit; all hypothesis-test statistics (Holm-adjusted p , bootstrap intervals) appear in the main-body results and the artifact.

B Localization Statistics

Trace-level pooled Fisher (descriptive only). Pooling the two paired stacks as if independent gives C07 4/6 vs. 0/6 ($p \approx 0.030$) and C17-I03 4/4 vs. 0/4 ($p \approx 0.014$); these treat dependent observations as independent and are *not* evidence (the seed-level values in Section 5.3 are inferential). **Sparse-failure rule.** At the seed level where it fires the rule is high-precision but sparse (C07 precision 1.00, recall 2/3; C17-I03 1.00, 2/2; zero false positives; fires in only two of 96/72 cells); full precision/recall/Wilson-CI tables are in the artifact. As an attack it would presuppose workflow/model identity the A0/A2 attacker lacks, so the localization is an offline auditor method.

Table 6. Extension (6 workflows), A0 and within-workflow A2. Most screen-positive cells are isolated and not learner-robust; the one configuration repeating across both stacks is A2/kind/exact/logistic (0.601, 0.613), reported as exploratory (many cells, $n = 6$ units, paired stacks, kind-aware). None is a confirmed attack.

Atk	Obs.	View	Stack 1		Stack 2	
			logreg	gbt	logreg	gbt
A0	safe	exact	0.548 i	0.550 T	0.560 T	0.526 i
A0	safe	bucketed	0.540 i	0.508 e	0.563 i	0.517 i
A0	safe	removed	0.564 i	0.462 i	0.566 T	0.491 i
A0	kind	exact	0.529 i	0.521 i	0.547 i	0.555 T
A0	kind	bucketed	0.557 T	0.508 i	0.566 i	0.505 i
A0	kind	removed	0.564 i	0.504 i	0.566 T	0.493 i
A2	safe	exact	0.561 i	0.581 i	0.569 i	0.498 i
A2	safe	bucketed	0.557 i	0.513 i	0.533 i	0.558 i
A2	safe	removed	0.595 i	0.526 i	0.618 i	0.582 T
A2	kind	exact	0.601 T	0.588 T	0.613 T	0.502 i
A2	kind	bucketed	0.575 i	0.533 i	0.545 i	0.544 i
A2	kind	removed	0.595 i	0.530 i	0.618 i	0.582 T

C Richer-Observer Secondary-Analysis Grid

Pre-declared design (from Section 7). Before viewing any result we fixed the features, learners, splits, and screen. Two observers are declared: an *identity-safe* observer using the generic `step.role` category only, and a *kind-aware* observer adding the operation kind (`tool.category/span.kind`); raw `op_id`, `tool.name`, and `phase` are excluded as workflow-specific. From these we compute count vectors, ordered category n -grams, per-category durations, token and status/retry/failure counts, and shallow tree statistics, and evaluate two frozen learners (standardized logistic regression and gradient-boosted trees). The *descriptive screen* is fixed in advance (mean signed AUC ≥ 0.55 with bootstrap lower bound > 0.5), an unadjusted secondary threshold, not the confirmatory rule. Whether the kind-aware observer is a legitimate threat model depends on whether the sanitizer exposes generic kinds or workflow-specific identities; we report both and never select the larger.

Signed deployable AUC means with verdicts (**i**=inconclusive, **e**=equivalent to chance at ± 0.05 TOST, **T**=meets the unadjusted 0.55 screen); “safe”=identity-safe, “kind”=kind-aware observer. For Experiment 1 (24 workflows, A0) all 24 configurations are inconclusive or equivalent to chance (mean 0.493, range 0.435–0.532), so we omit that all-null table and report only the extension grid, where the one configuration screen-positive in both stacks (A2/kind/exact/logistic, 0.601/0.613) is not learner-robust and is reported as hypothesis-generating.

References

1. Liu, G., Solomon, S.: AI Agent Observability: Evolving Standards and Best Practices. OpenTelemetry Blog (2025).
[urlhttps://opentelemetry.io/blog/2025/ai-agent-observability/](https://opentelemetry.io/blog/2025/ai-agent-observability/)
2. AlSayyad, A., Huang, K.Y., Pal, R.: AgentTrace: A Structured Logging Framework for Agent System Observability. arXiv:2602.10133 (2026)
3. Debenedetti, E., et al.: Privacy Side Channels in Machine Learning Systems. In: USENIX Security (2024)
4. Song, L., et al.: The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems. arXiv:2409.20002 (2024)
5. Zheng, X., et al.: InputSnatch: Stealing Input in LLM Services via Timing Side-Channel Attacks. arXiv:2411.18191 (2024)
6. Zhang, T., Saileshwar, G., Lie, D.: Time Will Tell: Timing Side Channels via Output Token Count in Large Language Models. arXiv:2412.15431 (2024)
7. McDonald, G., Bar Or, J.: Whisper Leak: A Side-Channel Attack on Large Language Models. arXiv:2511.03675 (2025)
8. Aerni, M., Zhang, J., Tramèr, F.: Evaluations of Machine Learning Privacy Defenses are Misleading. In: ACM CCS (2024)
9. Hayes, J., Danezis, G.: k-fingerprinting: A Robust Scalable Website Fingerprinting Technique. In: USENIX Security (2016)
10. Cherubin, G., Jansen, R., Troncoso, C.: Online Website Fingerprinting: Evaluating Website Fingerprinting Attacks on Tor in the Real World. In: USENIX Security, pp. 753–770 (2022)
11. Shao, Y., Li, T., Shi, W., Liu, Y., Yang, D.: PrivacyLens: Evaluating Privacy Norm Awareness of Language Models in Action. In: NeurIPS Datasets and Benchmarks (2024)
12. Zharmagambetov, A., Guo, C., Evtimov, I., Pavlova, M., Salakhutdinov, R., Chaudhuri, K.: AgentDAM: Privacy Leakage Evaluation for Autonomous Web Agents. In: NeurIPS Datasets and Benchmarks (2025)
13. Wang, S., Yu, F., Liu, X., Qin, X., Zhang, J., Lin, Q., Zhang, D., Rajmohan, S.: Privacy in Action: Towards Realistic Privacy Mitigation and Evaluation for LLM-Powered Agents. In: Findings of EMNLP (2025)
14. Wang, B., He, W., Zeng, S., Xiang, Z., Xing, Y., Tang, J., He, P.: Unveiling Privacy Risks in LLM Agent Memory. In: ACL (2025)
15. Carlini, N., Chien, S., Nasr, M., Song, S., Terzis, A., Tramèr, F.: Membership Inference Attacks From First Principles. In: IEEE Symposium on Security and Privacy (2022)
16. OpenTelemetry Authors: Handling Sensitive Data. OpenTelemetry Security Documentation (2026).
[urlhttps://opentelemetry.io/docs/security/handling-sensitive-data/](https://opentelemetry.io/docs/security/handling-sensitive-data/)